

Architecture and Design Specification For



12/03/2024

Team Members: Danny Elhamalawy, Kevin Gawinauth, Alex Pateroulakis, Elijah Philip,
Gayathri Suresh

Version 2.0

TABLE OF CONTENTS

1. Introduction:	3
1.1 Purpose:	3
1.2 Project Scope:	3
2. System Overview:	4
2.1 High-Level Overview:	4
2.2 Technical Characteristics:	5
2.3 Architecture Styles:	6
3. Key Architectural Components:	7
3.1 Programming Environment:	7
3.2 Database:	7
3.3 Email Notification:	8
3.4 Product Details:	8
3.5 Payment Processing:	9
4. Architecture and Design Patterns:	10
4.1 Repository:	10
4.2 Client-Server:	11
4.3 Observer Pattern:	13
4.4 Layered Architecture:	14
4.5 Pipe and Filter:	15
5. Functionality Designs:	17
5.1 Product Design:	17
5.2 Cart Design:	17
5.3 Product Categories:	18
5.4 Location Management:	19

1. Introduction:

The GreenGrocer app is a web-based grocery shopping application that allows users to browse different product categories, view product details, and manage their shopping carts to purchase products from local grocery stores. The app is designed with Bootstrap, HTML structure, and Python code to provide a simple yet effective graphic interface for users to select and purchase items such as fruits, meats, drinks, and frozen foods. The application integrates the Kroger API to fetch real-time grocery store data, providing users with up-to-date product information, including availability, pricing, and item details from Kroger's vast inventory. This app is designed to improve the grocery shopping experience by offering convenience and efficiency through unique and easy-to-navigate features.

1.1 Purpose:

This paper guides the architecture and design specifications modeled in the GreenGrocer grocery shop application's framework. The grocery app contains a platform allowing users to interact with the Kroger API to retrieve pricing and product information while viewing store locations, perusing product categories, adding items to a cart, and managing their favorites. The program supports basic e-commerce features like cart management and user authentication while managing a multi-user accessible interface.

1.2 Project Scope:

The project scope is building a user-friendly online application that interfaces with the Kroger API to receive and display product and price information particular to individual stores. The web application is available to anyone using a smartphone or computer with an Internet connection, allowing them to proceed through the grocery services as a guest or returning user. The application allows users to register, access, and control their accounts. Other functions include locating nearby store locations and their respective inventory, editing and managing in a cart, keeping track of user-favorite products, and viewing product availability and pricing variations between Kroger store locations.

2. System Overview:

2.1 High-Level Overview:

The Green Grocer application implements a Model-View-Controller or MVC pattern as the overarching architectural design. Functions and subsystems within the application use other design patterns that ultimately contribute to the part of the overall MVC model. The “Model” portion is the part of the system that stores data and handles API calls. JSON files maintain user account data to protect sensitive information and avoid unwanted manipulation. The API calls fetch data from the various APIs implemented in the system and manage the data along with the data’s associated operations. The “View” component is responsible for managing the presentation of the data and system to the user. This component correlates with the graphic user interface (GUI) that is rendered and maintained by HTML, JS, and CSS pages. The GreenGrocer application displays different interfaces to users based on their user type: registered user, unregistered user, and guest. All users will have access to the basic functionalities of the application, however, guests will have limited access to premium features (refer to Section 2.3 in the Green Grocer Requirements Document for further details). Ultimately, the “Controller” part of the architecture pattern manages the interaction of users with the application and passes on the requests as well as reflects the resulting changes in the View and Model components. This component is the implementation of Flask in the software product. Flask essentially acts as a server to handle requests and communicate with HTML and JSON files. This component is essential to integrate the front and backends of the overall system.

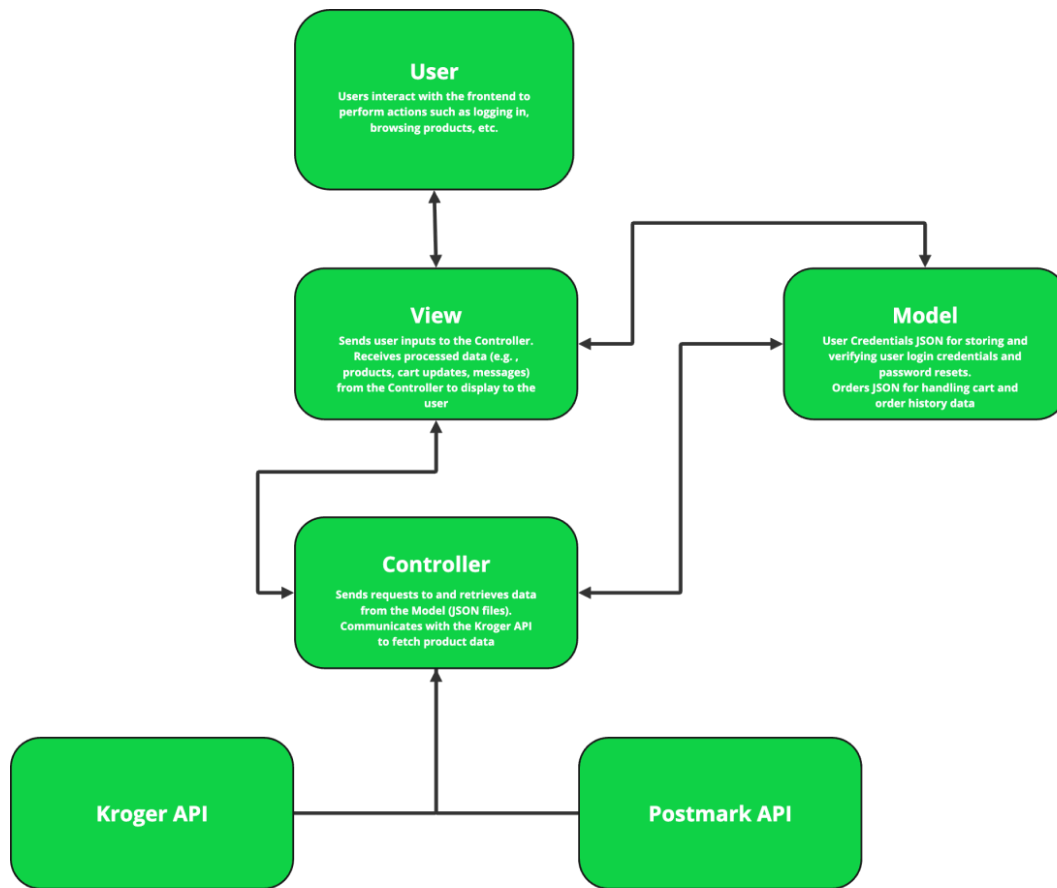


Figure 1. Diagram of Green Grocer’s System Modeled in the MVC Architecture Pattern. *The MVC diagram represents the interactions of the key components of the Green Grocer system involving user input and API integration.*

2.2 Technical Characteristics:

- **Frontend:** The Green Grocer application allows users to navigate a modern user-friendly design created by HTML, Bootstrap, JS, and CSS. This combination allows for a highly intuitive, responsive, and visually appealing interface, allowing users to seamlessly navigate features and functionalities. The design emphasizes interactive elements, and accessible layouts that enhance the overall user experience.
- **Backend:** The backend of the web application is based in Python with Flask as the main framework to handle the application. In conjunction with Flask, the JSON files, Kroger

API and the majority of the application features can run on a URL for local testing and final product deployment.

- **Database Storage:** User data, including credentials and account information, is stored in one JSON file, while a separate JSON file is dedicated to order history and order-specific details for each user. This alternative strategy is implemented to reduce the complexity and overhead associated with a full-scale database.
- **API Integration:** The Kroger API and Postmark API are integral to the features and characteristics of the application.

2.3 Architecture Styles:

The GreenGrocer web-based application implements an MVC architecture pattern as described in Section 2.1. As mentioned before, subsystems within the overall application employ other design patterns to support the available functionalities. One function that is important to the application is Product Categories. This feature is implemented via a pipe-and-filter design pattern, allowing users to view appropriate data from the huge influx of information available. The client-server design pattern is integrated into the system, mainly with the use of different APIs and different clients accessing the incoming real-time information concurrently, specifically in the display of product information and categories. Other architectural and design patterns modeled in the Green Grocer application are repository architecture for software version control using GitHub, observer patterns reflecting changes in the product and the cart, and layered architecture to protect API call IDs and backend information from unauthorized access.

3. Key Architectural Components:

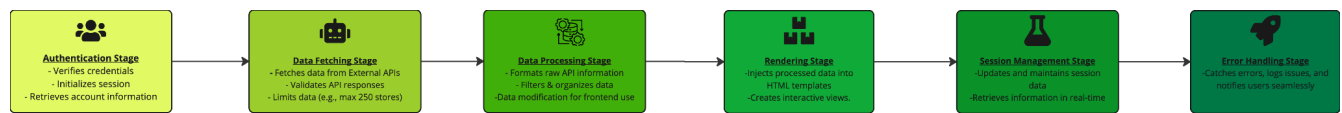


Figure 2. Pipeline Diagram of Data Flow within Green Grocer Application. *The pipeline diagram represents the several stages at which the user's actions are processed into steps for certain tasks within the application. These stages include view rendering of the HTML, session management, data processing, authentication, error handling, etc.*

3.1 Programming Environment:

The GreenGrocer app is developed using Visual Studio Code, a lightweight and accessible programming environment, to streamline development and testing. Github repositories were used for version control and collaboration. Each team member had their branch to develop specific components. Once the features were completed, these branches were merged into the main branch. The application is developed using Python, leveraging the Flask framework, which serves as the primary server environment for managing backend processes and handling HTTP requests. JSON is used as a lightweight database for handling user and order information. The front end of the application is built with HTML5, CSS, JavaScript, and Bootstrap to provide a modern and responsive user interface. Flask also serves as the backend web server, handling data processing, API integrations, and user session management.

3.2 Database:

The GreenGrocer app uses JSON files as the primary data storage solution to simplify the development process and avoid the overhead of setting up a relational database. JSON allows quick data access, essential during early development while offering flexibility for handling user and order information. This setup includes:

- User Data JSON: Stores essential user information, including usernames, hashed passwords, and user preferences.
- Order History JSON: This service maintains records of past orders for registered users, including product details, quantities, and dates.

While JSON provides a straightforward way to store and retrieve data during early development, it introduces concurrent access and scalability limitations. If the app grows beyond its initial user base, the JSON storage can be replaced with a relational database system like MySQL or PostgreSQL to handle larger datasets and improve query performance.

3.3 Email Notification:

The GreenGrocer app integrates an email notification system to keep users informed of important updates, account activities, and user verification, such as password resets, order confirmations, digital receipts, and user authentication. To achieve this functionality, user email addresses are retrieved from the JSON-based data store, and email content is dynamically generated from the backend. The Postmark API handles the email delivery process, ensuring reliable and secure transmission. This email notification system supports a responsive user experience while maintaining a high standard of email deliverability and sender reputation.

3.4 Product Details:

The GreenGrocer app integrates the Kroger API to access a comprehensive catalog of grocery products, presenting users with a rich selection of items, each complete with detailed descriptions and attributes. The product data provided by the Kroger API includes images of the items for users to view. Additionally, the API supplies up-to-date pricing information, which keeps product listings accurate. The API also includes categorical data, enabling better organization of products for easier navigation and searchability within the app.

3.5 Payment Processing:

The web application uses a mock transactions system to simulate payment processing. This allows users to go through purchasing steps without a real transaction. The fields that users would have to fill include the name on the card, card number, expiry date, CVV, shipping address, city, and zip code. The payment and location details are validated for formatting and ensure the entered shipping address exists. If the validation fails the system will notify users with clear, specific error messages underneath the input fields that were incorrectly filled. If the validation is successful the system will provide an order confirmation that will be emailed to the user through the Postmark API.

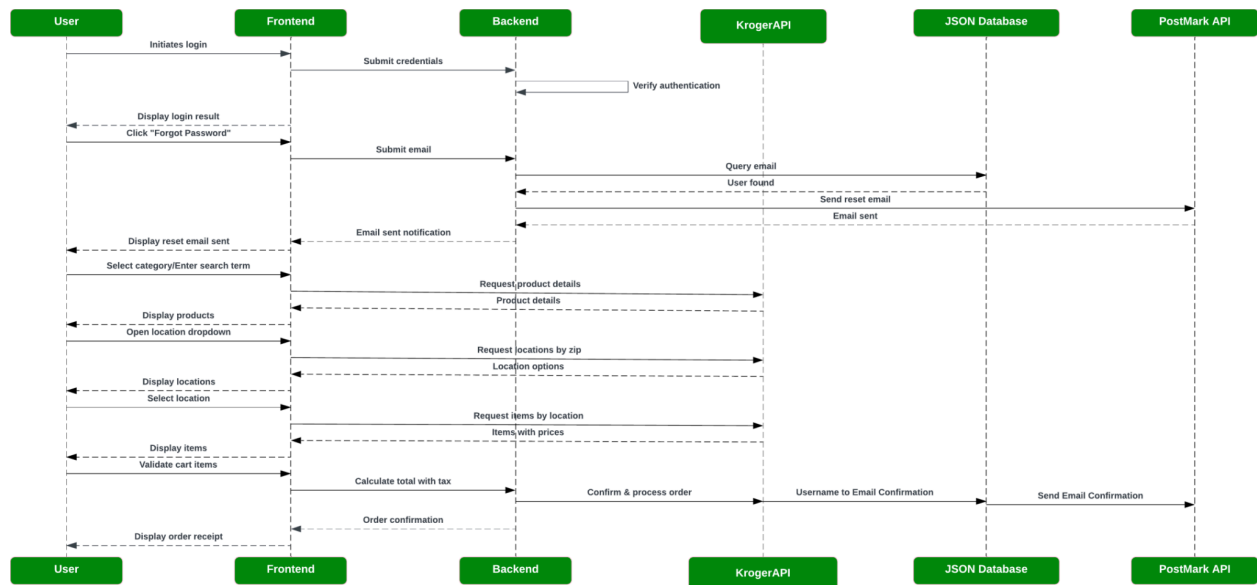


Figure 3. Sequence Diagram of Interaction of Components from MVC Architecture. *The diagram summarizes the interactions between the key architectural components of the GreenGrocer Application from Login actions and onwards.*

4. Architecture and Design Patterns:

4.1 Repository:

The GitHub repository is structured to streamline the app development with the following files being layered.

4.1.1 Root Directory:

/app:

- app.py: Acts as the main entry point and controller of the application. This file defines key route handlers and manages the backend logic.
- config.py: Stores environment-specific settings and configurations, such as API keys, debug settings, and database connection strings. This separation of configuration ensures that sensitive information remains secure and that different environments (development, testing, production) can be managed effectively.
- user_storage.json: Used as a lightweight database for storing user information. Enables rapid development and testing without the need for complex database setup in the initial stages.

/templates:

- Contains all HTML, JavaScript, and CSS templates, which are rendered dynamically through the Jinja2 templating engine integrated with Flask. By separating the HTML, Javascript, and CSS templates from the backend code, there's a clear separation between the front end from the back end, improving code readability and allowing front-end developers to work independently on UI elements.

/static:

- Stores static assets like images and visual elements. Organizing static resources here allows them to be served directly to users, optimizing load time and performance.

/docs:

- Holds all documentation resources, including architecture diagrams, API specifications, and testing reports. This ensures that all documentation is

centralized, allowing developers to quickly reference project guidelines, and improving consistency across the team.

4.1.2 Version Control:

- Branches: The repository uses a main branch for stable code and separate feature branches for individual development tasks. This branching strategy allows developers to work on isolated features without affecting the main codebase, reducing conflicts and ensuring that good quality tested code is merged.
- GitHub Actions: GitHub Actions are configured to automate continuous integration (CI) and continuous deployment (CD) processes. Every pull request triggers a series of automated tests, ensuring that new code integrates smoothly with the existing codebase and that errors are caught early. This CI/CD setup improves reliability, speeds up the feedback loop, and reduces the risk of regressions.
- Deployment: Supports Ngrok deployment, ensuring a seamless transition of updates. Ngrok enables the team to deploy the latest features with minimal downtime, and by using Ngrok's environment-specific settings, developers can handle different stages of deployment (staging and production) efficiently.

4.2 Client-Server:

The client (frontend) and the server (backend) are the two primary components of the program while following the architecture of a client-server based on the application below. Different user types can only view certain displays of the user interface. As a client, the users can interact with the server on their browser and all of its features which include product searches, category filtering, and the recently implemented feature location selection.

4.2.1 Frontend Client:

- User Interface: The client is the interface that users interact with directly through a browser or URL. It is built using JavaScript for interactivity, CSS for styling, and HTML for structure.
- User Experience: Users can access features like login, sign-up, search, location selection, cart, and checkout. The front end sends HTTP requests to the backend to retrieve data and displays the results to the user.
- Client-Server Responsiveness: The frontend sends API calls to the backend to fetch products that match a user's request, and then displays the results.

4.2.2 Backend Server:

All user requests are handled by the server, which uses Flask. It manages sessions, routes, and HTTP requests. Utilizing Flask allows it to process requests and control the application by defining routes for activities within the coding application environment. Say, for example, product search(/products) or login(/login). It allows users to remain signed in throughout the application by managing user sessions and validating data such as their login credentials.

- Kroger API: Flask is essential for using the Kroger API because it retrieves the product and location information from the API. Flask is the tool that allows the clients to communicate with the servers and APIs.
- Postmark API: Flask also communicates with Postmark to send emails by using Flask Mail which allows to confirm orders and help reset passwords within the application.

- Security Protection: Flask uses features such as data encryption and validation to integrate session management to safely handle sensitive data, such as login credentials.

4.3 Observer Pattern:

GreenGrocer employs the observer pattern to ensure that users receive real-time updates and notifications as they interact with the application. This pattern facilitates communication between the application's backend and the user interface, establishing a responsive UI.

- Real-Time Notifications: The observer pattern allows for immediate feedback in response to user actions. For instance, once a user is authenticated, any changes they make to their profile are reflected immediately in the user interface.
- Cart Management: As users add or remove items from their cart, the cart counter and stored cart information are automatically updated in real time. This ensures that users have an accurate view of their cart's contents without needing to refresh the page, thus facilitating a smoother shopping experience.
- Location Selection Updates: When a user selects their preferred store location, all pages that rely on location-specific data (such as product listings, special pricing, and product availability) are dynamically updated. This means that users will see tailored content based on their selected location.
- Enhanced User Experience: The implementation of the observer pattern improves the responsiveness of the application and also fosters a more interactive and engaging environment for users. This design choice ensures that users are consistently informed about the state of their interactions, ultimately leading to increased satisfaction and engagement with the platform.

4.4 Layered Architecture:

The GreenGrocer App is designed with a layered architecture to enhance modularity, security, and scalability. This architecture includes three distinct layers: the Presentation Layer, the Application Layer, and the Data Access Layer. The Presentation Layer, or Frontend, consists of HTML templates and static resources like CSS and JavaScript, providing a responsive and engaging user interface. The Application Layer, managed by `app.py` handles routing, session tracking, and core application logic, ensuring secure and seamless user interactions. Finally, the Data Access Layer manages secure data storage, CRUD operations, and API integrations, enabling users to access essential information, including API client IDs, without risking data exposure.

The layered approach adds an extra layer of security by preventing unauthorized access across layers; users cannot bypass one layer to access another directly. This layered security prevents potential data breaches, as each layer acts as a checkpoint, ensuring only authorized requests progress through the system. For example, the API client ID is an important and private component to ensure the full functionality of the application. Therefore, if users can obtain the system's client ID, then they can access the backend and tamper with data that is integral to the basic processes. With the implementation of the layered architecture, users only have access to the front layer and may only be able to communicate with the middle layer. In this way, the API handling is kept secure and hidden from direct user access. The information from API calls is available to users indirectly via proxying but prevents direct manipulation and data leakage. This design promotes a safe, organized, and reliable user experience on the GreenGrocer platform.

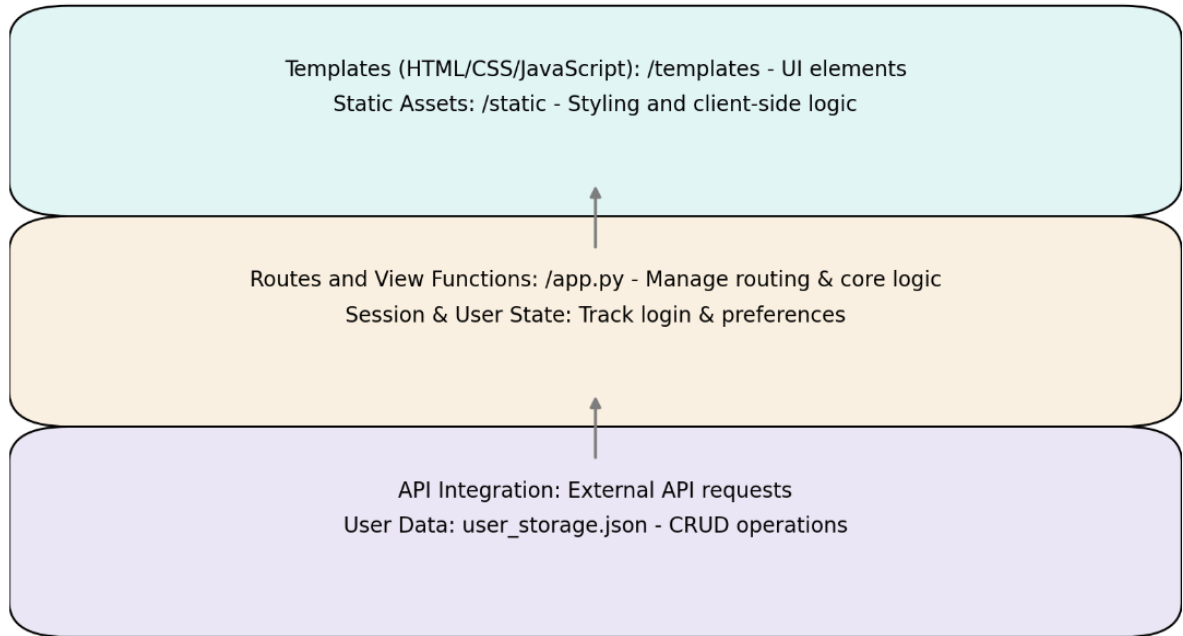


Figure 4. Layered Diagram containing the Component Layers in the GreenGrocer application. *Arrows indicate the movement of data from the API backend to the frontend user interface through the middle layer. Layers may only directly communicate with layers they are in contact with, otherwise proxying needs to take place to execute requests.*

4.5 Pipe and Filter:

GreenGrocer utilizes a pipe and filter design pattern to curate the product list given to the user based on their store location and product category selection.

- Input Pipe: The user inputs their store location and selects their desired product category from the application interface (Home Page). This information is sent through the input pipe, initiating the data processing workflow. Currently, the location filter only allows users to choose from stores within Chicago to ensure the application works in one location before expanding.
- Location Filter: The first filter in the pipeline analyzes the user's store location. This filter ensures that the subsequent steps only consider relevant products from the store the user selects. Another feature shows the user if a product is in stock at another nearby location

if it's not in stock at the store they selected. Additionally, the location filter allows to display/inform the user if a certain product is discounted.

- Product Category Filter: The product category filter narrows down the results further. It cross-references the available products at the identified store with the user's selected category. For example, if the user wants to purchase chicken, only the valid products in stock at this store are shown. This allows for a more targeted product list that meets the user's needs.
- Output Pipe: The filtered product list is then passed to the output pipe, which formats and presents the data in the user-friendly UI.
- Scalability and Maintenance: The use of the pipe and filter design pattern enhances the scalability and maintainability of the GreenGrocer application. Each filter can be independently modified or replaced without affecting the overall system, allowing for easy updates or integration of new features, such as adding additional filters based on user preferences (e.g., organic products, promotions, etc.).

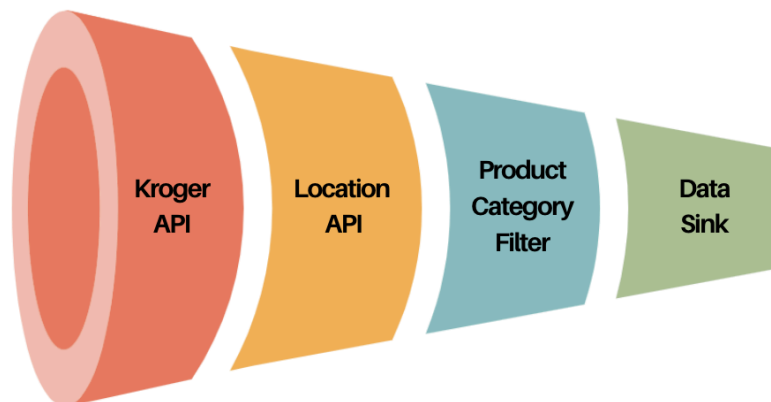


Figure 5. Pipe and Filter Diagram illustrating the data flow in the GreenGrocer application. *The Kroger API serves as the initial data source for the location filter, which processes location-based queries. The output from the location filter then feeds into the product filter, allowing for refined product selection based on user preferences. Finally, the filtered data is presented to the user, facilitating a personalized shopping experience.*

5. Functionality Designs:

This section highlights important design decisions made to shape the important functionalities of the GreenGrocer application. The Green Grocer Requirements Document provides further information regarding the requirements and functionality of the features mentioned below.

5.1 Product Design:

The Kroger API calls back images for each specific product being searched. To display the products simply, a card-based pattern is used. The pattern offers the image of the item being shown, the name of the item, and an add-to-cart button with pricing based on the selected location. In addition to these features, there is a button on each item to allow the user to easily add a product to their Favorites list.



Simple Truth® All Natural Boneless Skinless Fresh Chicken Breast Tenders

Price: \$7.49

Add to Cart

Figure 6. Example product page of a chicken product from the Meats category

5.2 Cart Design:

A list-view pattern with interactive buttons was chosen as the design for the cart since the design focuses on showing the product image and information. This allows the user to easily review their order before purchase. It also allows the user to quickly increase and decrease the amount of each item purchased without having to navigate to the individual item page. Lastly, there is a remove

item button on each item and a save for later option towards the bottom of the Cart for ease of use.

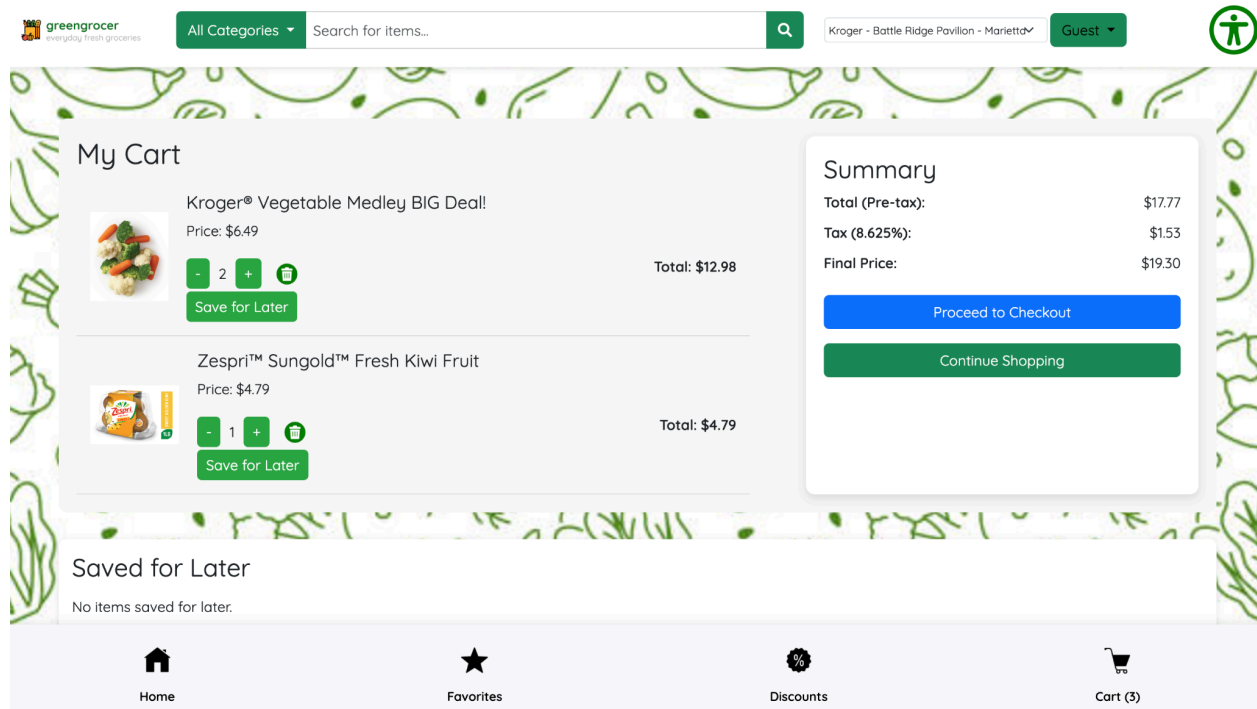


Figure 7. The following Image shows the GreenGrocer’s cart in its current implementation. *Users can interact with a counter to increment or decrease the quantity, removing the item completely and a save for later features.*

5.3 Product Categories:

The product categories feature is implemented as a dropdown menu next to the product search bar, allowing users to filter their search easily. The decision to place the categories button was on purpose to create an intuitive and easy-to-navigate user interface. The user can select whichever category they wish to browse and either scroll through the products of that subgroup or use the search bar to look for a specific brand name or item within the category. This feature allows users to find their product of interest faster and more efficiently compared to scrolling through and scouring through the whole inventory.

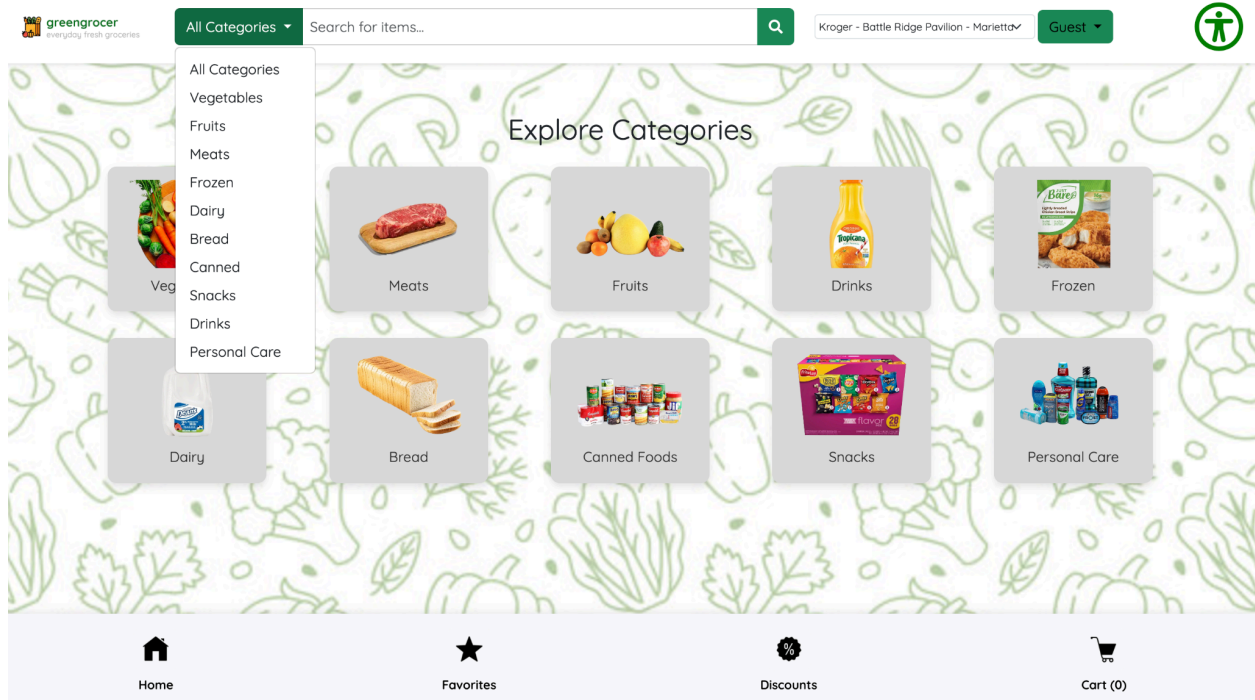


Figure 8. Current Green Grocer Design for the Categories Feature. *The Search functionality bar features a drop-down box for product filtering in certain categories or all categories at once for ease of access.*

5.4 Location Management:

The GreenGrocer App allows users to select a specific location to shop from, which is essential as inventory availability varies by location. Users can access available locations through a dropdown menu labeled “Select Location” located next to the search bar. This feature ensures that users view accurate inventory based on their chosen location, enhancing the shopping experience by providing relevant and localized product options. The available locations to select from are all Kroger’s locations within a certain radius. The dropdown design allows for easy navigation and search for locations based on the user’s preferences.

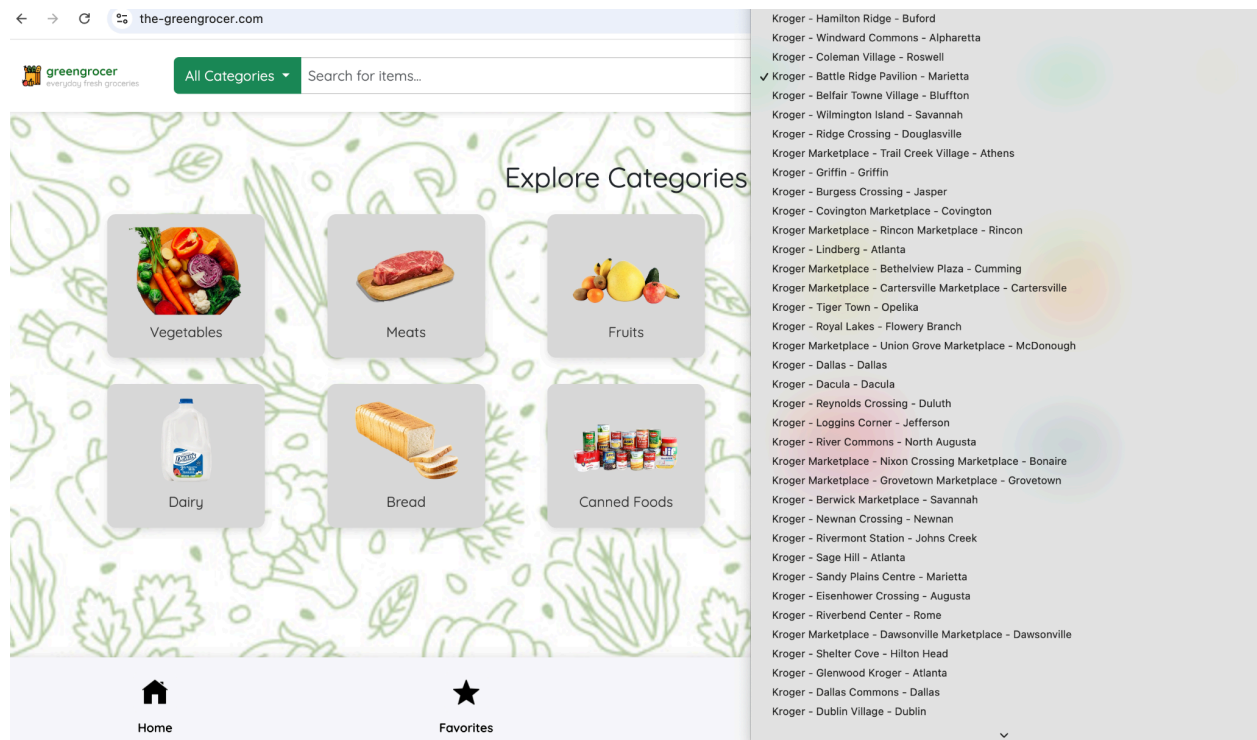


Figure 9. Current Green Grocer Design for Location Management. *The locations are displayed in a Drop-Down Box on the GreenGrocer Home Page.*